

Laporan Praktikum Kontrol Cerdas Week 4

Nama : Cecillia Yowien Arthamevia
NIM : 224308031
Kelas : TKA-6B
Akun Github : <https://github.com/CecilliaYowienArthamevia>
Student Lab Assistant : Salma Halizah Zuhroh

1. Judul Percobaan: *Reinforcement Learning for Autonomous Control*

2. Tujuan Percobaan:

Tujuan dari percobaan *reinforcement learning for autonomous control* sebagai berikut:

- Memahami konsep dasar *Reinforcement Learning* (RL) dalam sistem kendali.
- Mengimplementasikan agen RL menggunakan algoritma *Deep Q-Network* (DQN)
- Menggunakan *OpenAI Gym* sebagai simulasi lingkungan untuk pelatihan RL.
- Melatih dan menguji agen RL untuk mengontrol lingkungan secara otonom.
- Menggunakan GitHub untuk version control dan dokumentasi praktikum.

3. Landasan Teori:

Reinforcement Learning (RL) adalah metode pembelajaran mesin yang memungkinkan agen belajar melalui interaksi dengan lingkungan menggunakan sistem penghargaan (*reward*) dan hukuman (*punishment*). *Reinforcement Learning* (RL) sering digunakan dalam berbagai aplikasi yang membutuhkan pengambilan keputusan secara adaptif dan optimal dalam lingkungan yang dinamis (Irvansyah, 2020).

Reinforcement Learning (RL) didasarkan pada interaksi antara agen dan lingkungan, di mana agen mengambil tindakan berdasarkan keadaan saat ini dan menerima umpan balik dalam bentuk reward atau punishment. Pendekatan ini sangat cocok untuk masalah yang sulit dipecahkan dengan pemrograman konvensional, seperti sistem kendali otonom, permainan video, dan optimasi lalu lintas (Irvansyah, 2020).

Dalam sistem kendali, *Reinforcement Learning* (RL) diterapkan untuk mengoptimalkan pengambilan keputusan secara real-time tanpa memerlukan model eksplisit dari sistem yang dikendalikan. Salah satu contoh penerapan RL dalam sistem kendali adalah optimasi arus lalu lintas perkotaan, dimana agen RL dapat belajar pola lalu lintas dan menyesuaikan pengaturan sinyal lalu lintas untuk mengurangi kemacetan (Fatih dkk., 2024).

Selanjutnya pada praktikum kali ini menggunakan *Deep Q-Network* (DQN) yaitu pengembangan dari metode *Q-Learning* yang menggunakan jaringan saraf tiruan (*Deep Neural Networks*) untuk memetakan keadaan ke nilai tindakan yang optimal. DQN bekerja dengan menyimpan pengalaman dalam *replay buffer* dan memperbarui model berdasarkan sampel pengalaman yang diambil secara acak, sehingga meningkatkan stabilitas dan efisiensi pembelajaran. Algoritma ini telah diterapkan dalam berbagai skenario, seperti pengaturan lampu lalu lintas adaptif, yang menunjukkan kinerja lebih baik dibandingkan metode konvensional berbasis waktu tetap (Putra dkk., 2024).

Selain itu pada praktikum ini juga menggunakan OpenAI Gym yang merupakan pustaka yang digunakan untuk mengembangkan dan menguji algoritma *Reinforcement Learning* (RL) dalam lingkungan simulasi yang standar dan terstruktur. Gym menyediakan berbagai lingkungan simulasi, termasuk sistem fisika dan robotika, yang mempermudah penelitian dan perbandingan algoritma *Reinforcement Learning* (RL). Dengan menggunakan OpenAI Gym, pengembang dapat menguji dan melatih model *Reinforcement Learning* (RL) dalam lingkungan yang terkendali sebelum mengimplementasikannya dalam aplikasi dunia nyata (Irvansyah, 2020).

4. Analisis dan Diskusi:

- Analisis

Pada praktikum minggu keempat ini percobaan yang dilakukan adalah membuat *Reinforcement Learning for Autonomous Control* dengan mengimplementasikan agen RL menggunakan algoritma *Deep Q-Network* (DQN) dan *OpenAI Gym* sebagai simulasi lingkungan untuk pelatihan RL. Dimana pada percobaan praktikum ini, performa agen dalam mengontrol environment *CartPole* dianalisis menggunakan algoritma *Deep Q-Network* (DQN). Agen belajar untuk menyeimbangkan tiang pada tumpuan dengan mengambil tindakan yang optimal berdasarkan nilai *Q-value* yang diperoleh selama pelatihan. Hasil eksperimen menunjukkan bahwa agen mampu meningkatkan waktu keseimbangan tiang secara signifikan setelah beberapa iterasi pelatihan. Seiring dengan bertambahnya episode, agen menunjukkan peningkatan dalam kemampuannya mengontrol lingkungan, ditandai dengan peningkatan skor rata-rata dan stabilitas keputusan yang lebih baik. Namun, dalam tahap awal pelatihan, agen sering kali gagal menjaga keseimbangan karena kebijakan awal masih bersifat eksploratif dan belum menemukan strategi optimal.

Selain itu perubahan parameter seperti *gamma*, *epsilon*, dan *learning rate* berpengaruh signifikan terhadap kinerja agen. Parameter *gamma* menentukan seberapa jauh agen mempertimbangkan reward di masa depan, nilai *gamma* yang lebih tinggi akan mendorong agen untuk mempertimbangkan keputusan jangka panjang, sedangkan nilai yang lebih rendah akan membuat agen lebih fokus pada reward langsung. Sementara itu, *epsilon* dalam *epsilon-greedy policy* mengontrol keseimbangan antara eksplorasi dan eksploitasi. Pada awal pelatihan, nilai *epsilon* yang tinggi diperlukan untuk memungkinkan agen menjelajahi berbagai kemungkinan tindakan, tetapi jika nilainya terlalu tinggi dalam jangka panjang, agen dapat terus mengambil keputusan suboptimal. Di sisi lain, *learning rate* mempengaruhi kecepatan pembaruan parameter jaringan saraf. Jika *learning rate* terlalu besar, agen bisa mengalami ketidakstabilan dalam

pembelajaran karena perubahan parameter yang terlalu drastis. Sebaliknya, apabila *learning rate* terlalu kecil, agen membutuhkan waktu yang lebih lama untuk mencapai performa optimal.

Kemudian terdapat beberapa tantangan utama yang muncul selama pelatihan agen RL meliputi *overfitting*, ketidakstabilan pembelajaran, dan kesulitan dalam menemukan kebijakan optimal. Agen mungkin mengalami *overfitting* jika hanya belajar dari sejumlah kecil pengalaman dan tidak memiliki cukup variasi dalam data pelatihan. Selain itu, ketidakstabilan pembelajaran sering terjadi akibat korelasi tinggi dalam sampel data atau karena pembaruan bobot jaringan saraf yang terlalu agresif. Penggunaan *experience replay* dapat membantu mengatasi masalah ini dengan menyimpan dan mengacak pengalaman sebelum digunakan untuk pembaruan. Selain itu, menemukan kebijakan optimal dalam *CartPole* memerlukan penyesuaian parameter yang cermat, karena lingkungan yang sensitif terhadap perubahan kebijakan dapat menyebabkan agen gagal mempertahankan keseimbangan dalam waktu lama. Dengan tuning parameter yang tepat dan jumlah episode pelatihan yang cukup, agen dapat mencapai performa yang optimal dalam mengontrol environment *CartPole*.

- Diskusi

Pada praktikum minggu keempat ini dalam percobaan yang telah dilakukan dapat disimpulkan terdapat perbedaan utama antara *Reinforcement Learning (RL)* dan metode *Supervised Learning* dalam sistem kendali yaitu terletak pada cara pembelajaran dilakukan. *Supervised Learning* memerlukan data pelatihan yang telah diberi label sebelumnya, di mana model belajar dari pasangan *input-output* yang sudah diketahui untuk memprediksi atau mengklasifikasikan data baru. Sebaliknya, *Reinforcement Learning* tidak memiliki target yang jelas di awal dan belajar melalui interaksi langsung dengan lingkungan, menerima *reward* berdasarkan tindakan yang dilakukan. Dalam sistem kendali, metode *Supervised Learning* lebih cocok untuk tugas yang memiliki hubungan input-output yang sudah diketahui, seperti pengenalan pola atau klasifikasi

sinyal sensor. Sementara itu, *RL* lebih efektif dalam skenario yang memerlukan pengambilan keputusan secara sekuensial dan adaptif, seperti kontrol robotik atau optimasi jalur kendaraan otonom.

Strategi eksplorasi (*exploration*) dan eksploitasi (*exploitation*) dalam *RL* harus dioptimalkan agar agen dapat menemukan kebijakan terbaik untuk pengambilan keputusan. Eksplorasi memungkinkan agen mencoba berbagai tindakan untuk menemukan strategi yang lebih baik, sedangkan eksploitasi memanfaatkan pengetahuan yang telah diperoleh untuk memaksimalkan *reward*. Salah satu metode yang umum digunakan adalah *epsilon-greedy policy*, di mana agen memilih tindakan secara acak dengan probabilitas *epsilon* dan memilih tindakan terbaik berdasarkan nilai *Q-value* dengan probabilitas $(1-\epsilon)$. Nilai *epsilon* biasanya dikurangi secara bertahap selama pelatihan untuk meningkatkan eksploitasi setelah agen memiliki pengalaman yang cukup. Alternatif lainnya adalah *Upper Confidence Bound (UCB)* dan *Thompson Sampling*, yang lebih adaptif dalam menyeimbangkan eksplorasi dan eksploitasi berdasarkan ketidakpastian tindakan yang diambil.

Potensi aplikasi lain dari *RL* dalam sistem kendali nyata sangat luas dan mencakup berbagai bidang industri. Dalam robotika, *RL* dapat digunakan untuk pelatihan robot agar dapat beradaptasi dengan lingkungan yang dinamis, seperti navigasi robot dalam ruang yang tidak terstruktur atau manipulasi objek dengan lengan robotik. Dalam transportasi, *RL* telah diterapkan dalam sistem pengaturan lalu lintas adaptif yang dapat mengoptimalkan sinyal lampu lalu lintas berdasarkan kepadatan kendaraan. Selain itu, dalam sistem tenaga listrik, *RL* digunakan untuk optimasi distribusi daya dan kontrol jaringan pintar (*smart grid*). Aplikasi lain termasuk pengelolaan energi dalam bangunan pintar, pengendalian drone otonom, dan sistem rekomendasi yang dapat menyesuaikan preferensi pengguna berdasarkan interaksi sebelumnya.

5. Assignment :

Dalam praktikum minggu keempat ini percobaan yang dilakukan adalah membuat *Reinforcement Learning for Autonomous Control* dengan mengimplementasikan agen RL menggunakan algoritma *Deep Q-Network* (DQN) dan *OpenAI Gym* sebagai simulasi lingkungan untuk pelatihan RL. Dimulai dengan implementasi agen *Deep Q-Network (DQN)* dalam lingkungan *CartPole* menggunakan OpenAI Gym yang terdiri dari beberapa tahap utama, yaitu pembuatan agen, pelatihan agen, dan pengujian agen. Pada tahap pembuatan agen mendefinisikan kelas dengan kode program **class DQNAgent**, dimana kode program tersebut merepresentasikan agen yang akan belajar mengontrol lingkungan. Dalam kelas ini memiliki beberapa komponen utama diantaranya inisialisasi agen dengan kode program **def __init__**. Didalam inisialisasi agen terdapat kode program **self.memory = deque(maxlen=2000)** merupakan buffer yang menyimpan pengalaman agen untuk pelatihan, **self.gamma = 0.95** untuk menentukan seberapa jauh agen mempertimbangkan *reward*, **self.epsilon = 1.0** digunakan dalam strategi eksplorasi-exploitation, **self.learning_rate = 0.001** untuk kecepatan pembelajaran dari model. Selanjutnya membangun model DQN dengan kode program **def _build_model** yaitu model menggunakan jaringan saraf tiruan dengan tiga dense layers, dimana pada layers pertama digunakan untuk menerima input berupa *state* dari lingkungan, lapisan kedua adalah lapisan tersembunyi untuk ekstraksi fitur, lapisan ketiga menghasilkan nilai Q untuk setiap aksi, dan model dioptimasi dengan menggunakan **Adam Optimizer** dengan fungsi kehilangan **Mean Squared Error (MSE)**. Tahap selanjutnya adalah pelatihan agen DQN dalam lingkungan *CartPole* dengan pendekatan pengambilan tindakan dengan menggunakan kode program **act**, dimana agen akan memilih secara acak (eksplorasi) atau berdasarkan modelnya (eksploitasi). Selanjutnya menyimpan pengalaman dengan menggunakan kode program **remember**, dimana setiap pengalaman (*state, action, reward, next_state, done*) disimpan dalam **memory buffer** untuk pelatihan. Kemudian memperbarui model dengan menggunakan kode program **replay**, agen akan mengambil

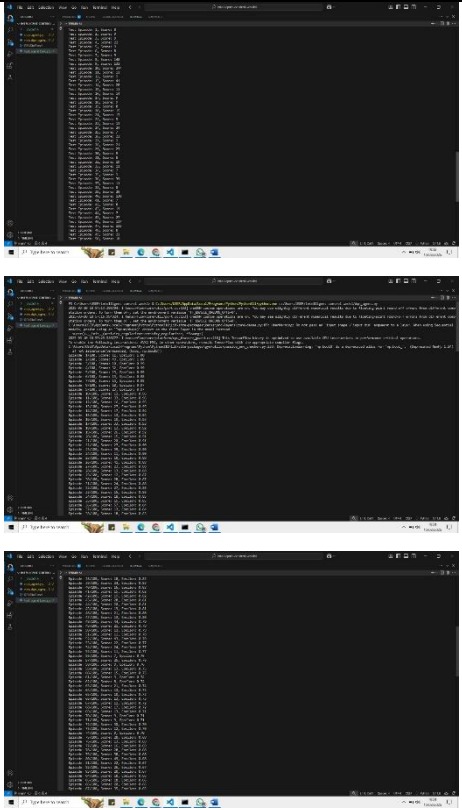
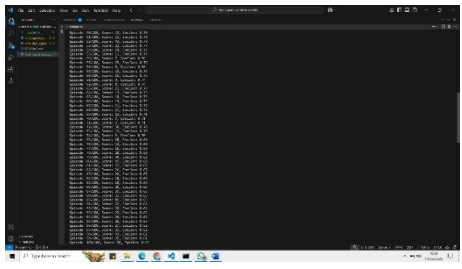
batch pengalaman secara acak dari *memory* dan memperbarui modelnya dan nilai *epsilon* dikurangi secara bertahap agar agen lebih banyak mengeksploitasi keputusan terbaiknya. Lalu Loop Pelatihan Agen dengan menggunakan kode program **range**, dimana agen akan mereset lingkungan di setiap episode, melakukan aksi berdasarkan kebijakan DQN dan memperbarui *memory*. Jika agen gagal menjaga keseimbangan tiang, maka akan diberikan pinalti (-10), dimana agen telah dilatih secara bertahap dengan menggunakan fungsi **replay ()** jika *memory* cukup besar.

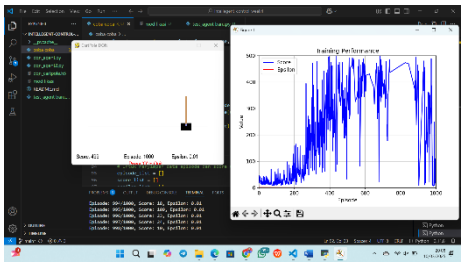
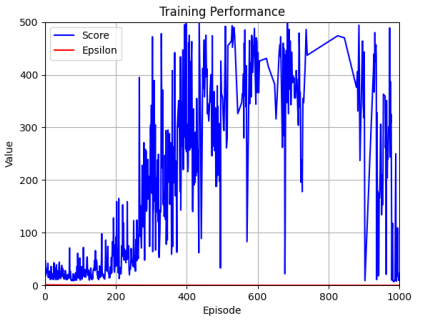
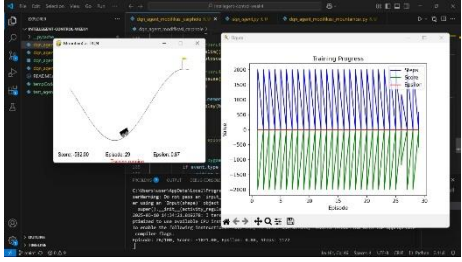
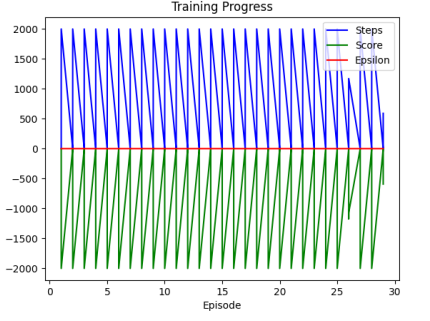
Tahap selanjutnya setelah agen dilatih maka dilakukan pengujian dan visualisasi agen yang dimulai dengan inisialisasi dan pengujian agen. Dengan nilai **epsilon** diatur rendah agar agen lebih banyak mengeksplorasi strategi yang optimal. Kemudian dilakukan Loop Pengujian dengan menggunakan fungsi **for e in range()**, dimana agen akan diuji dalam episode yang diminta dan hasilnya akan divisualisasikan menggunakan fungsi **env.render()**, dimana performa agen akan diukur berdasarkan skor (jumlah langkah sebelum jatuh).

Dalam praktikum yang telah dilakukan pada minggu ke empat ini mengenai modifikasi program dengan menambahkan teknik **target network** untuk meningkatkan stabilitas pelatihan program serta menguji agen pada *environment* lain seperti **mountaincar-v0** untuk dibandingkan performanya. Dengan mengimplementasikan agen *Reinforcement Learning* (RL) menggunakan algoritma Deep Q-Network (DQN) dan menambahkan visualisasi grafis dari lingkungan **CartPole-v1** serta meningkatkan stabilitas pelatihan program pada *environment* **MountainCar-v0** sistem memiliki kelebihan pada visualisasi *real-time* menggunakan **Pygame**, yang memberikan pemahaman intuitif tentang interaksi agen dengan lingkungan, serta plotting grafik dinamis dengan **matplotlib** yang memantau kemajuan pelatihan. Penggunaan **Keras** dan **TensorFlow** mempermudah pembangunan dan pelatihan model jaringan saraf tiruan, sementara *memory replay* dan strategi *epsilon-greedy* meningkatkan stabilitas dan efisiensi pembelajaran. Namun, kode ini juga memiliki kekurangan, yaitu

kompleksitas visualisasi yang dapat memperlambat pelatihan dan menambah beban komputasi, ketergantungan pada pustaka eksternal yang beragam, dan waktu pelatihan yang relatif lama, terutama untuk lingkungan yang kompleks seperti **MountainCar-v0**.

6. Data dan Output Hasil Pengamatan:

No	Program	Percobaan	Output
1.	Menggunakan kode program dqn_agent.py tanpa modifikasi	Percobaan yang dilakukan dengan menggunakan kode program dqn_agent.py dengan 100 episode tanpa menampilkan visualisasinya	
2.	Menggunakan kode program test_agent.py	Percobaan yang dilakukan dengan menggunakan kode program test_agent.py dengan 50 episode tanpa menampilkan visualisasinya.	

3.	Menggunakan kode program Reinforcement learning_CartPole.py (sudah dimodifikasi cartpole)	Percobaan yang dilakukan dengan menggunakan kode program Reinforcement learning_CartPole.py dengan 1000 episode serta menampilkan gambar visualisasinya dan menyajikan data berupa grafik.	 
4.	Menggunakan kode program Reinforcement learning_MountainCar-v0.py (sudah dimodifikasi dengan mountaincar)	Percobaan yang dilakukan dengan menggunakan kode program Reinforcement learning_MountainCar-v0.py dengan 1000 episode serta menampilkan gambar visualisasinya dan menyajikan data berupa grafik.	 

7. Kesimpulan:

Berdasarkan praktikum dan analisis yang telah dilakukan, maka dapat diambil kesimpulan yaitu:

- Agen *Reinforcement Learning (RL)* dapat belajar melalui interaksi dengan lingkungan menggunakan algoritma *Deep Q-Network (DQN)*, dimana DQN mampu meningkatkan performa agen dengan memanfaatkan pengalaman yang disimpan dalam *replay buffer* dan memperbarui bobot jaringan saraf secara bertahap.
- Agen RL diuji dalam lingkungan simulasi *CartPole* yang disediakan oleh OpenAI Gym, dimana agen harus menyeimbangkan tiang pada tumpuan.
- Hasil pelatihan menunjukkan bahwa agen dapat meningkatkan skor dan mempertahankan keseimbangan lebih lama seiring dengan bertambahnya episode pelatihan.
- Variabilitas dalam lingkungan simulasi dapat menyebabkan agen mengalami kesulitan dalam menemukan strategi yang dapat diterapkan secara konsisten di berbagai kondisi.
- RL dengan DQN memiliki potensi besar untuk diterapkan dalam sistem kendali otonom seperti robotika, kendaraan otonom, pengaturan lalu lintas adaptif, dan manajemen energi dalam *smart grid*.

8. Saran:

Untuk meningkatkan efektivitas praktikum pada minggu ke empat yaitu *Reinforcement Learning for Autonomous Control* dengan algoritma *Deep Q-Network (DQN)* dan OpenAI Gym. Terdapat beberapa saran yang dapat dilakukan untuk mengatasi tantangan yang muncul selama proses percobaan. Yang pertama yaitu pemilihan dan penyesuaian parameter hiper (*hyperparameter tuning*) seperti *gamma*, *epsilon decay*, *learning rate*, dan ukuran *replay buffer* perlu dilakukan secara sistematis untuk menemukan konfigurasi yang optimal bagi agen dalam pembelajaran dan eksploitasi strategi terbaik. Selanjutnya penggunaan teknik stabilisasi pembelajaran seperti *target network* yang diperbarui secara periodik dan implementasi

metode *prioritized experience replay* dapat membantu mengatasi ketidakstabilan pembaruan bobot jaringan saraf. Kemudian untuk menghindari *overfitting* dan meningkatkan generalisasi agen RL, eksperimen dapat mencakup variasi lingkungan latihan, seperti perubahan parameter lingkungan atau gangguan (*noise*) pada observasi agen. Selain itu, penggunaan algoritma *deep reinforcement learning* yang lebih canggih, seperti *Double DQN* atau *Dueling DQN*, dapat diuji untuk membandingkan performanya dengan DQN standar dalam mengatasi eksplorasi berlebihan dan konvergensi yang lambat. Kemudian peningkatan kompleksitas simulasi dengan lingkungan yang lebih realistis atau penerapan pada perangkat keras fisik seperti robot otonom akan memberikan wawasan lebih lanjut mengenai kesiapan algoritma RL dalam skenario dunia nyata. Dengan menerapkan saran-saran ini, praktikum diharapkan dapat memberikan pemahaman yang lebih mendalam tentang strategi optimasi RL dan aplikasinya dalam sistem kendali otonom.

9. Daftar Pustaka:

- Chen, W.-H. (2022). Perspective view of autonomous control in unknown environment: Dual control for exploitation and exploration vs reinforcement learning. *Neurocomputing*, 497, 50–63. <https://doi.org/10.1016/j.neucom.2022.04.131>
- Fatih, M. A., Ramadhan, A. T., Pratama, N. A., & Aqil, A. (2024). *OPTIMALISASI ARUS LALULINTAS PERKOTAAN MENGGUNAKAN REINFORCEMENT LEARNING*. 2(3).
- Irvansyah, M. (2020). *IMPLEMENTASI METODE REINFORCEMENT LEARNING PADA SIMULASI PERGERAKAN ROBOT MULTI-SENDI*.
- Lei, L., Tan, Y., Zheng, K., Liu, S., Zhang, K., & Shen, X. (2020). Deep Reinforcement Learning for Autonomous Internet of Things: Model, Applications and Challenges. *IEEE Communications Surveys & Tutorials*, 22(3), 1722–1760. <https://doi.org/10.1109/COMST.2020.2988367>

Putra, R. A., Syahbana, Y. A., & Ananda. (2024). Implementasi Algoritma Deep Q-Network (DQN) pada Lampu Lalu Lintas Adaptif Berdasarkan Waktu Tunggu dan Arus Kendaraan. *The Indonesian Journal of Computer Science*, 13(5). <https://doi.org/10.33022/ijcs.v13i5.4372>